

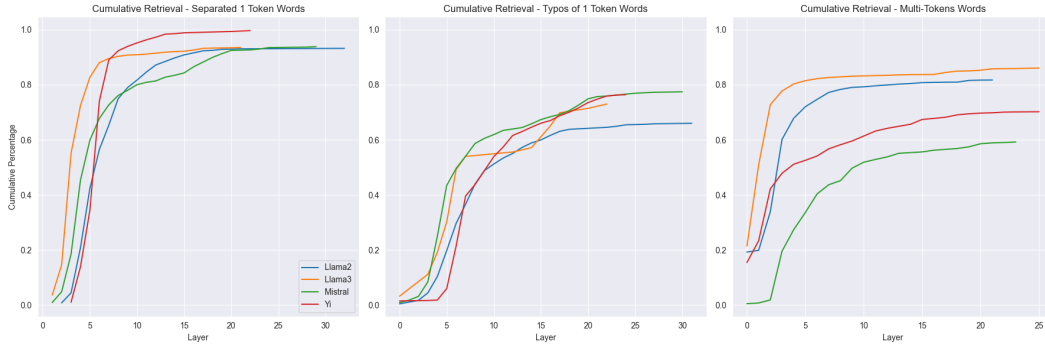


Figure 11: Cumulative word retrieval for single-token and multi-token words across all models.

The similarity in cumulative retrieval between single-token and multi-token words suggests that LLMs treat out-of-vocabulary words in a manner similar to sub-word-tokenized words, accessing a latent vocabulary to reconstruct full word representations.

D INTRODUCING TYPOS FOR SINGLE-TOKEN WORDS

In this section, we describe the process of introducing typos into single-token words to split them into multiple tokens (Sec. 4.1). The modification applies to words longer than four characters and involves randomly performing one of three operations: substituting two characters, deleting a character, or inserting a new character. By introducing these slight variations, the word becomes unfamiliar to the tokenizer, causing it to be divided into multiple smaller tokens during tokenization. Particularly, this process results in splitting words into 2–5 tokens. Table 4 shows examples of the different splits.

Description	perturbed	new tokens
Substitution of two characters	devel p oment	['de', 'vel', 'p', 'oment']
Deletion of one character	devel o ment	['de', 'vel', 'oment']
Insertion of one character	devel f opment	['dev', 'elf', 'op', 'ment']

Table 4: Examples of the different typos we consider, exemplified by perturbing the single-token word “development” (Sec. 4.1).

E ABLATION EXPERIMENT ON SUFFIX-SPLIT WORDS

In Sec. 5, to test the role of FFN layers in detokenization, we conduct an intervention-based experiment measuring how ablations to FFN updates affect word retrieval. We run this experiment on all single-token words from WIKITEXT-103 that end with one of three common suffixes: “*ing*,” “*ion*,” or “*est*”. Each word is then artificially split to two parts—the root word and the suffix. For example, the word “*eating*” is split into “*eat*” and “*ing*”, while “*connection*” is split into “*connect*” and “*ion*”. This ensures that processing the suffix token is necessary to reconstruct the identity of the word, and reduces possible effects of strong distributional artifacts of token co-occurrence.

Using logit lens, we identify FFN layers where the update to the residual stream can be decoded as the original single-token word. We then ablate these layers by zeroing out their updates to the residual stream. As a control, we ablate the same proportion (5%) of random FFN layers.

Our results, shown in Fig. 12, reveal that ablating the identified FFN layers lead to a sharp drop in word retrieval rates, effectively disrupting the detokenization process. In contrast, ablating random layers has little to no effect on retrieval accuracy. This indicates that the FFN layers play a critical role in reconstructing word representations rather than simply enhancing contextualization.

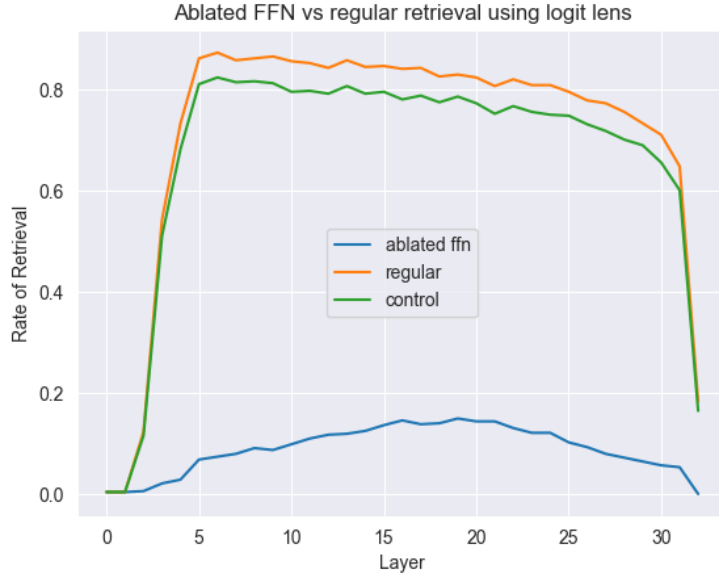


Figure 12: Comparison of retrieval rates using logit lens for suffix-split words under three conditions: ablation of identified FFN layers (blue line), regular retrieval (orange line), and control with random FFN layer ablation (green line). Ablation of critical layers causes a sharp drop in retrieval accuracy, highlighting their importance in detokenization.

F COMPARISON BETWEEN LOGIT LENS AND COSINE SIMILARITY

To validate our use of the logit lens, we repeat the artificial split-word experiment using cosine similarity. We use the same setting of artificial splits based on suffixes detailed above in App. E.

For the logit lens, we adapt the standard implementation to measure similarity between the hidden representation of the final token and the input embedding space (vocabulary space), rather than the output embedding space typically used to predict the next token. This adjustment allows us to directly evaluate the alignment between the hidden states and the embeddings of the original words in the vocabulary. In parallel, cosine similarity is used to compute the similarity between the same hidden states and the embeddings of the original words in the vocabulary space.

Our results, shown in Fig. 13, demonstrate that both methods produce nearly identical patterns. In both cases, retrieval accuracy peaks in the middle layers, where the hidden representation of the suffix token aligns most closely with the original word. Beyond these middle layers, retrieval accuracy declines, likely reflecting the model’s shift toward representing the prediction of the next token rather than maintaining the full representation of the current word.

These findings reinforce the validity of using the logit lens when adapted to the input embedding space, as a streamlined approach for analyzing model representations. This approach results in similar trends to cosine similarity while offering a more interpretable and direct framework for measuring alignment with the vocabulary space.

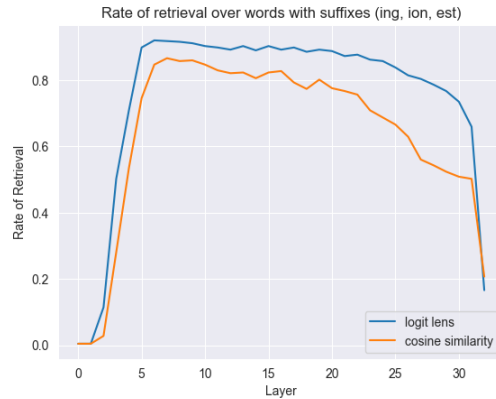


Figure 13: Comparison of retrieval accuracy for split-word experiments using either logit lens (blue line) and cosine similarity (orange line) across model layers.

G LEARNING THE LINEAR MAPS $T_{\ell,E}$ AND $T_{\ell,U}$

To expand the model’s vocabulary without modifying its core parameters, we construct linear transformations that map hidden states at different layers to the model’s embedding and unembedding spaces. By learning these transformations solely from the model’s existing vocabulary, we can infer new token representations without modifying any of the model’s core weights. This section details our method for learning these mappings.

G.1 EXTRACTING MODEL REPRESENTATIONS

Given a pretrained language model with input embedding matrix $E \in \mathbb{R}^{V \times d}$ and output unembedding (LM head) matrix $U \in \mathbb{R}^{V \times d}$, where V is the vocabulary size and d is the hidden dimension, we extract the representations used for learning the transformations as follows:

- **Embedding and Unembedding Matrices:** We extract the model’s embedding matrix E and LM head matrix U before making any modifications to the vocabulary.
- **Hidden States Across Layers:** For each token t in the model’s original vocabulary, we pass t as a single-token input and record its hidden state at every layer of the model. Let $h_\ell(t)$ denote the hidden state of token t at layer ℓ .

G.2 LEARNING THE LINEAR MAPPINGS

For each layer ℓ , we aim to learn two linear transformations:

- $T_{\ell,E}$ that maps hidden states to input embeddings.
- $T_{\ell,U}$ that maps hidden states to unembedding representations.

We learn these mappings as **orthogonal Procrustes problems** (Schönemann, 1966), which seek to find the best orthogonal transformation aligning two sets of vectors. Specifically, for each layer ℓ , we solve:

$$T_{\ell,E} = \arg \min_T \sum_{t \in V} \|Th_\ell(t) - e_t\|^2, \quad \text{subject to } T^\top T = I \quad (1)$$

$$T_{\ell,U} = \arg \min_T \sum_{t \in V} \|Th_\ell(t) - u_t\|^2, \quad \text{subject to } T^\top T = I \quad (2)$$

where e_t and u_t are the embedding and unembedding vectors of token t , respectively. The constraints enforce that each transformation preserves distances and does not distort the structure of the space. In our experiments, we use the Python implementation of Meng et al. (2022a).

G.3 NORMALIZATION WITH RMS SCALING

To preserve the relative magnitudes of embedding and unembedding entries, we normalize all representations using their **root mean square (RMS) norm** (Zhang & Sennrich, 2019). Specifically:

1. **Preprocessing Training Representations:** Before fitting the Procrustes transformations, we normalize all hidden states, embeddings, and unembedding vectors by dividing each vector x by its RMS norm:

$$x_{\text{norm}} = \frac{x}{\|x\|_{\text{RMS}}}, \quad \text{where } \|x\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}. \quad (3)$$

2. **Applying the Learned Maps:** When using the trained transformations on new detokenized representations r , we apply the following steps:

- (a) Normalize r by its RMS norm: $r_{\text{norm}} = \frac{r}{\|r\|_{\text{RMS}}}$.
- (b) Apply the learned transformation: $\hat{e} = T_{\ell,E} r_{\text{norm}}$, $\hat{u} = T_{\ell,U} r_{\text{norm}}$.
- (c) Rescale by the mean RMS of the target space:

$$e = \hat{e} \cdot \mathbb{E}[\|e_t\|_{\text{RMS}}], \quad u = \hat{u} \cdot \mathbb{E}[\|u_t\|_{\text{RMS}}]. \quad (4)$$